

CloudWatch and X-Ray

Metrics · Logs · Alarms · Dashboards · Distributed Tracing · Container Insights · ShopFlow

AWS Tutorial · ShopFlow

14-01

Why Observability Matters for ShopFlow

■ Observability: Metrics, Logs, and Traces

Observability is the ability to understand what your application is doing right now and why it behaved that way in the past. AWS provides three overlapping tools: CloudWatch (metrics and logs), X-Ray (distributed traces), and CloudWatch Logs Insights (log analysis). Together they form the observability stack for ShopFlow.

Example: Black Friday 2024: ShopFlow checkout latency spiked to 8 seconds at 9 PM. Without observability, it would take hours to find the cause. With CloudWatch + X-Ray, the team identified in 4 minutes that DynamoDB was throttling order writes — the root cause was a missing GSI. Fix deployed in 12 minutes.

Pillar	AWS Tool	What You See	ShopFlow Example
Metrics	CloudWatch Metrics	Numeric time-series data: counts, rates, percentages	ECS CPU, ALB Request count, DynamoDB consumed
Logs	CloudWatch Logs	Timestamped text output from applications and services	Spring Boot application logs, ALB access logs, Lambda
Traces	AWS X-Ray	End-to-end request journey across multiple services	Customer clicks item to cart: API Gateway -> ECS -> Dy
Dashboards	CloudWatch Dashboards	Combined view of metrics in one place	ShopFlow Ops Dashboard: ALB RPS + ECS CPU + Au
Alarms	CloudWatch Alarms	Alert when metric crosses threshold	ALB 5xx > 1% for 5 minutes -> SNS -> PagerDuty alert

14-02

CloudWatch Core Concepts

CloudWatch organizes metric data into namespaces, metrics, and dimensions. Understanding this structure lets you query exactly the data you need.

Concept	Definition	ShopFlow Example
Namespace	Top-level category for metrics — usually the AWS service name	AWS/ECS (CPU metrics), AWS/ApplicationELB (ALB metrics), Shop
Metric	Named time-series within a namespace	CPUUtilization (in AWS/ECS), RequestCount (in AWS/ApplicationEL
Dimension	Key-value filter that identifies which specific resource the metric belongs to	ClusterName=shopflow-prod, ServiceName=product-svc (filters CPU
Period	Time granularity of data points	60 seconds = one data point per minute; 300s (5-min) for dashboard
Statistic	Aggregation function applied to data points in a period	Average, Sum, Maximum, Minimum, p99 (99th percentile), SampleC
Retention	How long CloudWatch stores metric data	1-second metrics: 3 hours; 60-second metrics: 15 days; 5-minute me

14-03

CloudWatch Console — Finding ShopFlow ECS Metrics

Follow these steps to find ShopFlow ECS CPU and memory metrics in the CloudWatch Console.

■ Navigate to ECS Metrics in CloudWatch

1. Open AWS Console -> Search "CloudWatch" -> CloudWatch
2. Left sidebar -> Metrics -> All metrics
3. In the "Browse" tab, click "ECS" under AWS namespaces
4. Click "ClusterName, ServiceName" to see per-service metrics
5. Type "shopflow-prod" in the filter box to narrow results
6. Check "CPUUtilization" and "MemoryUtilization" for "product-svc"
7. Click "Graphed metrics" tab -> Change Period to 1 Minute
8. Change Statistic to "Average" for CPU, "Maximum" for Memory
9. Click "Actions" -> "Add to dashboard" to save this view

■ aws > CloudWatch > Metrics > All metrics > ECS > ClusterName, ServiceName

ECS Service Metrics — shopflow-prod

■ Search Metrics

Filter metrics

shopflow-prod

■ Metric Results (ClusterName: shopflow-prod)

- CPUUtilization — ServiceName: product-svc | Namespace: AWS/ECS | Unit: Percent
- MemoryUtilization — ServiceName: product-svc | Namespace: AWS/ECS | Unit: Percent
- CPUUtilization — ServiceName: order-svc | Namespace: AWS/ECS | Unit: Percent
- MemoryUtilization — ServiceName: order-svc | Namespace: AWS/ECS | Unit: Percent

■ Graph Options

Period

1 Minute ■

Statistic

Average ■

Time Range

Last 3 hours ■

Add to Dashboard

14-04

Custom Metrics — Business KPIs from Spring Boot

AWS services publish their own metrics automatically. But ShopFlow also needs business metrics: orders per minute, cart abandonment rate, and search latency percentiles. These are published to CloudWatch using the AWS SDK from the Spring Boot application.

```
// Spring Boot: Publish custom metrics to CloudWatch @Component public class ShopFlowMetrics {  
    private final CloudWatchAsyncClient cw = CloudWatchAsyncClient.create(); private static final
```

```
String NAMESPACE = "ShopFlow/Orders"; // Call after successful order placement public void
recordOrderPlaced(String productCategory, double orderValueUsd) {
    cw.putMetricData(PutMetricDataRequest.builder() .namespace(NAMESPACE) .metricData(
        MetricDatum.builder() .metricName("OrdersPlaced") .value(1.0) .unit(StandardUnit.COUNT)
        .dimensions(Dimension.builder() .name("Category").value(productCategory).build())
        .timestamp(Instant.now()) .build(), MetricDatum.builder() .metricName("OrderValueUSD")
        .value(orderValueUsd) .unit(StandardUnit.NONE) .dimensions(Dimension.builder()
        .name("Category").value(productCategory).build()) .timestamp(Instant.now()) .build()
    ).build()); } // Call on cart abandonment (user had items but did not check out after 30 min)
public void recordCartAbandonment(int itemCount) {
    cw.putMetricData(PutMetricDataRequest.builder() .namespace(NAMESPACE)
        .metricData(MetricDatum.builder() .metricName("CartAbandonment") .value((double) itemCount)
        .unit(StandardUnit.COUNT) .timestamp(Instant.now()) .build()) .build()); } }
```

INFO Micrometer + Spring Boot Actuator (Preferred Approach)

In production, ShopFlow uses Micrometer (included in Spring Boot Actuator) with the CloudWatch Metrics Registry (micrometer-registry-cloudwatch2). This is simpler: annotate methods with `@Timed`, `@Counted`, or use the `MeterRegistry` directly. Micrometer batches and publishes metrics to CloudWatch automatically every 60 seconds. The raw SDK approach above shows what Micrometer does under the hood.

14-05

CloudWatch Alarms — Getting Paged Before Customers Complain

Alarms monitor a single metric and trigger an SNS notification when the metric crosses a threshold for N consecutive periods. ShopFlow has alarms for every critical failure mode.

■ Create an Alarm for High ALB 5xx Error Rate

1. CloudWatch -> Alarms -> All Alarms -> Create alarm
2. Select metric -> ApplicationELB -> Per AppELB Metrics -> HTTPCode_ELB_5XX_Count
3. Filter: LoadBalancer = shopflow-prod-alb -> Select metric
4. Metric name: HTTPCode_ELB_5XX_Count | Statistic: Sum | Period: 1 minute
5. Conditions: Threshold type = Static | Greater than = 100
6. Datapoints to alarm: 3 out of 3 (alert only if 3 consecutive minutes > 100 5xx errors)
7. Notification: Select or create SNS topic "shopflow-alerts" -> Add action
8. Alarm name: ShopFlow-ALB-5xx-High | Description: ALB returning >100 5xx per minute for 3 min
9. Create alarm

Alarm	Metric	Threshold	Action
ALB 5xx High	HTTPCode_ELB_5XX_Count (Sum/1min)	> 100 for 3 consecutive minutes	SNS -> PagerDuty (page on-call)
ECS CPU High	CPUUtilization (Average/5min) per service	> 80% for 2 consecutive periods	SNS -> scale out ECS service (Auto Scaling)
Aurora Connections High	DatabaseConnections (Maximum/5min)	> 900 (max_connections = 1000)	SNS -> email + Slack alert
DynamoDB Throttled Requests	ThrottledRequests (Sum/1min)	> 0 for 5 consecutive minutes	SNS -> email SRE team (review capacity)
ElastiCache Evictions	Evictions (Sum/5min)	> 1000	SNS -> email (cache too small; increase size)
Order Count Drop	OrdersPlaced (Sum/5min) — custom metric	< 10 during business hours (9 AM-9 PM)	SNS -> PagerDuty (checkout may be broken)

14-06

CloudWatch Logs — Capturing Application Output

ECS Fargate tasks send application output (stdout/stderr) to CloudWatch Logs via the `awslogs` log driver. Spring Boot logs go here automatically when configured in the task definition.

```
// CDK: Configure CloudWatch Logs for ECS task LogGroup appLogs = LogGroup.Builder.create(this,
"AppLogs") .logGroupName("/ecs/shopflow/product-svc") .retention(RetentionDays.ONE_MONTH)
.removalPolicy(RemovalPolicy.DESTROY) .build(); ContainerDefinition container =
taskDef.addContainer("app", ContainerDefinitionOptions.builder()
.image(ContainerImage.fromEcrRepository(repo, "latest"))
.logging(LogDrivers.awsLogs(AwsLogDriverProps.builder() .logGroup(appLogs)
.streamPrefix("product-svc") .build())) .build());
```

With Spring Boot + Logback, configure JSON structured logging so CloudWatch Logs Insights can parse fields:

```
{"service":"product-svc","env":"prod"} // Output in CloudWatch Logs (each line is a JSON
object): // { // "timestamp": "2026-06-17T14:23:01.234Z", // "level": "ERROR", // "logger":
"com.shopflow.product.ProductService", // "message": "Product not found", // "productId":
"PROD-12345", // "traceId": "1-abc123-xyz456", // "service": "product-svc" // }
```

14-07

CloudWatch Logs Insights — Querying Your Logs

Logs Insights is a query language for CloudWatch Logs. Instead of scrolling through log streams, you write a SQL-like query to find errors, compute latencies, or count events.

■ Run a Logs Insights Query

1. CloudWatch -> Logs -> Logs Insights
2. Select log groups: /ecs/shopflow/product-svc and /ecs/shopflow/order-svc
3. Set time range: Last 1 hour
4. Type or paste your query in the query editor
5. Click "Run query" — results appear below in seconds
6. Click "Save" to save the query for later reuse
7. Click "Add to dashboard" to display query results on a CloudWatch Dashboard

```
-- Logs Insights: Find all errors in the last hour fields @timestamp, level, message, productId,
@logStream | filter level = "ERROR" | sort @timestamp desc | limit 50 -- Logs Insights: Measure
product search latency percentiles fields @timestamp, searchLatencyMs | filter message like
/ProductSearch completed/ | stats avg(searchLatencyMs) as avgMs, pct(searchLatencyMs, 95) as
p95Ms, pct(searchLatencyMs, 99) as p99Ms by bin(5m) -- Logs Insights: Count errors by product
category fields @timestamp, productCategory, level | filter level = "ERROR" | stats count(*) as
errorCount by productCategory | sort errorCount desc | limit 10 -- Logs Insights: Find orders
that took > 5 seconds end-to-end fields @timestamp, orderId, durationMs, @logStream | filter
message like /OrderPlaced/ and durationMs > 5000 | sort durationMs desc
```

14-08

CloudWatch Dashboards — ShopFlow Operations Center

A CloudWatch Dashboard is a single screen combining metrics graphs, alarm status, and Logs Insights queries. ShopFlow's on-call SRE has a dashboard open during business hours and Black Friday.

```
// CDK: Create ShopFlow Ops Dashboard dashboard = Dashboard.Builder.create(this,
"ShopFlowDashboard") .dashboardName("ShopFlow-Production-Ops") .build(); // Row 1: Traffic
overview (ALB) dashboard.addWidget( GraphWidget.Builder.create() .title("ALB Requests per
Second") .left(List.of(Metric.Builder.create() .namespace("AWS/ApplicationELB")
.metricName("RequestCount") .dimensionsMap(Map.of("LoadBalancer",
alb.getLoadBalancerFullName())) .statistic("Sum").period(Duration.minutes(1)).build()))
.build(), GraphWidget.Builder.create() .title("ALB Error Rate (4xx + 5xx)") .left(List.of(
Metric.Builder.create() .namespace("AWS/ApplicationELB") .metricName("HTTPCode_ELB_4XX_Count")
.statistic("Sum").period(Duration.minutes(1)).build(),
Metric.Builder.create() .namespace("AWS/ApplicationELB") .metricName("HTTPCode_ELB_5XX_Count")
.statistic("Sum").period(Duration.minutes(1)).build())) .build()); // Row 2: ECS Service Health
dashboard.addWidget( GraphWidget.Builder.create() .title("ECS CPU Utilization per Service")
.left(List.of( ecsCpuMetric("product-svc"), ecsCpuMetric("order-svc"),
ecsCpuMetric("cart-svc"))) .build(), SingleValueWidget.Builder.create() .title("Alarm Status")
.metrics(List.of(alarmMetric, albAlarmMetric)) .build());
```

■ Distributed Tracing with X-Ray

When a ShopFlow customer places an order, the request travels through multiple services: API Gateway -> order-svc ECS task -> Aurora (write) -> DynamoDB (cart clear) -> SNS (order event) -> SQS -> notification Lambda. X-Ray tracks this entire journey as a single "trace" with timing for each "segment" (service). The trace ID is passed between services via HTTP headers.

Example: Black Friday: Customer clicks "Place Order" -> X-Ray shows trace: API Gateway (3ms) + ECS order-svc (847ms) + Aurora write (12ms) + DynamoDB (820ms — THROTTLED). Root cause: DynamoDB write capacity exhausted. Without X-Ray this would have looked like a generic "slow ECS task."

Concept	Definition	ShopFlow Example
Trace	One end-to-end request journey from entry to exit	Trace ID: abc123-xyz: customer POST /orders from click to 200 OK
Segment	One service's contribution to the trace	order-svc segment: started 14:23:01.100, ended 14:23:01.947 (847ms)
Subsegment	A named sub-operation within a segment	DynamoDB subsegment: 12ms; order-svc -> DynamoDB subsegment
Service Map	Visual graph showing services and their connections	Nodes: API Gateway, order-svc, Aurora, DynamoDB, SNS, SQS, notification Lambda
Sampling rate	% of requests traced (tracing every request)	ShopFlow 5% sampling rate at normal traffic; 1% on Black Friday peak
Annotations	Indexed key-value pairs attached to traces	orderStatus: PLACED, requestId: 1234567890, userId: user123

ShopFlow uses the AWS X-Ray Java SDK with Spring Boot auto-instrumentation. Once configured, X-Ray automatically traces every incoming HTTP request and all AWS SDK calls (DynamoDB, SQS, S3, etc.) without modifying business logic.

```
// pom.xml: X-Ray dependencies for Spring Boot 3 + Java 21 // aws-xray-recorder-sdk-core,
aws-xray-recorder-sdk-spring, // aws-xray-recorder-sdk-aws-sdk-v2-instrumentor (all version
2.15.1) // Spring Boot configuration class for X-Ray @Configuration public class XRayConfig {
@Bean public Filter xRayFilter() { return new AWSXRayServletFilter("shopflow-order-svc"); } }
// Business logic: add custom annotations and metadata to the trace @Service public class
OrderService { public Order placeOrder(OrderRequest req) { Subsegment seg =
AWSXRay.beginSubsegment("placeOrder"); try { // Annotations are INDEXED – searchable in X-Ray
console seg.putAnnotation("orderId", req.orderId()); seg.putAnnotation("userId", req.userId());
seg.putAnnotation("productCategory", req.category()); // Metadata is NOT indexed – for
debugging only seg.putMetadata("orderRequest", req.toString()); Order order =
processOrder(req); seg.putAnnotation("orderStatus", "PLACED"); return order; } catch (Exception
e) { seg.addException(e); // marks subsegment as error in X-Ray throw e; } finally {
AWSXRay.endSubsegment(); } } }
```

The X-Ray console provides a Service Map and a Traces view. Use the Service Map to spot which service is slow, then drill into individual traces to find the exact operation causing the delay.

■ Find Slow Order Traces in X-Ray

1. AWS Console -> X-Ray -> Service map
2. Set time range: Last 1 hour
3. Look for orange/red nodes (high latency or errors); click "order-svc" node
4. Right panel shows: Latency histogram, Requests/min, Error rate for order-svc
5. Click "View traces" to see individual order traces
6. X-Ray -> Traces -> Filter: annotation.orderStatus = "FAILED" (only failed orders)
7. Sort by Duration (descending) to see slowest traces first
8. Click a slow trace -> Trace detail view shows flame-chart of all segments
9. Identify which segment/subsegment accounts for the most time

■ aws > X-Ray > Traces

ShopFlow Traces — Last 1 Hour

■ Filter Traces

Filter expression

annotation.productCategory = "Electronics" AND duration > 2

■ Trace Results (23 traces matching filter)

- Trace 1-6abc-def123 | Duration: 4.2s | Status: 200 | order-svc | 14:23:01
- Trace 1-6abd-fed456 | Duration: 3.8s | Status: 200 | order-svc | 14:21:44
- Trace 1-6abe-aaa789 | Duration: 2.3s | Status: 500 | order-svc | 14:19:22

■ Trace Detail — 1-6abc-def123 (4.2s total)

- API Gateway — 3ms
- shopflow-order-svc — 847ms
- Aurora INSERT orders — 12ms
- DynamoDB UpdateItem (cart clear) — 820ms [THROTTLED]
- SNS Publish (order.placed) — 8ms
- notification-lambda — 3.3s (cold start: 2.8s + execution: 0.5s)

14-12

X-Ray CDK Configuration — IAM and Sampling

X-Ray requires the ECS task IAM role to have permission to send trace data, and a sampling rule to control which requests are traced.

```
// CDK: Grant X-Ray permissions to ECS task role taskRole.addManagedPolicy(
ManagedPolicy.fromAwsManagedPolicyName("AWSXRayDaemonWriteAccess")); // CDK: Custom sampling
rule (5% of all requests, 100% of /orders POST) CfnSamplingRule checkoutRule =
CfnSamplingRule.Builder.create(this, "CheckoutSamplingRule")
.samplingRule(CfnSamplingRule.SamplingRuleProperty.builder()
.ruleName("ShopFlow-Checkout-Always") .priority(100) // lower number = higher priority
.reservoir(10) .fixedRate(1.0) // 100% sampling for checkout .host("") .httpMethod("POST")
```

```

.resourceArn("") .serviceName("shopflow-order-svc") .serviceType("") .urlPath("/orders")
.version(1) .build() .build(); CfnSamplingRule defaultRule =
CfnSamplingRule.Builder.create(this, "DefaultRule")
.samplingRule(CfnSamplingRule.SamplingRuleProperty.builder() .ruleName("ShopFlow-Default")
.priority(1000) .reservoir(5) .fixedRate(0.05) // 5% of all other requests
.host("").httpMethod("").resourceArn(""))
.serviceName("shopflow-*").serviceType("").urlPath("") .version(1).build() .build();

```

14-13

Container Insights — Deep ECS Metrics

Container Insights is a CloudWatch feature that enables detailed per-container metrics for ECS Fargate. Without Container Insights, you only get cluster-level metrics.

Metric	Without Container Insights	With Container Insights
CPU granularity	Cluster average only	Per task AND per container within the task
Memory usage	Task definition memory limit only	Actual RSS memory used by each container in real-time
Network I/O	Not available on Fargate	Bytes per second in/out per task
Storage I/O	Not available on Fargate	Read/write bytes per second for ephemeral storage
Cost	No extra charge	~\$12/month for 60 tasks (at \$0.50/M custom metrics)

```

// CDK: Enable Container Insights on ECS Cluster – single flag
Cluster cluster =
Cluster.Builder.create(this, "Cluster") .clusterName("shopflow-prod") .vpc(vpc)
.containerInsights(true) // enables all per-task metrics .build(); // CloudWatch publishes
metrics under: ECS/ContainerInsights namespace // Metrics: CpuUtilized, MemoryUtilized,
NetworkRxBytes, NetworkTxBytes, // StorageReadBytes, StorageWriteBytes, RunningTaskCount // Log
group created automatically: /aws/ecs/containerinsights/shopflow-prod/performance

```

14-14

ShopFlow Observability Architecture

Here is how all ShopFlow services connect to the observability stack.

ShopFlow Observability Stack

CloudWatch

Metrics

Logs

Alarms

Dashboards

AWS X-Ray

Traces — 5% sampling (100% for /orders)

Service Map — visual call graph with latency and error rates

Sampling Rules — ShopFlow-Checkout-Always (100%)

Container Insights

Per-task CPU, Memory, Network, Storage (60 Fargate tasks)

14-15

Hands-On Lab — Create Alarm and View X-Ray Trace

In this lab you will create a CloudWatch Alarm for ECS CPU and then generate and view an X-Ray trace. Time estimate: 30 minutes.

■ Lab: CloudWatch Alarm + X-Ray Trace

1. CloudWatch -> Alarms -> Create alarm -> Select metric -> ECS -> ClusterName,ServiceName
2. Pick CPUUtilization for your service, statistic Average, period 5 minutes
3. Threshold: Greater than 70 -> Datapoints to alarm: 2 out of 2
4. Create new SNS topic "test-alerts" with your email as subscriber
5. Confirm the subscription in your email inbox
6. Trigger it: `aws ecs update-service --cluster shopflow-dev --service product-svc --desired-count 0`
7. X-Ray -> Service Map -> Verify shopflow-order-svc appears
8. Generate a trace by calling POST /orders on the dev environment
9. X-Ray -> Traces -> Find the trace -> Click it -> Examine segments
10. Note the DynamoDB subsegment timing — is it under 10ms? If not, check for throttling
11. Cleanup: restore desired count, delete test alarm

14-16

Composite Alarms — Reduce Alert Fatigue

A Composite Alarm combines multiple alarms with AND/OR logic. ShopFlow uses composite alarms to reduce false alerts: page the on-call engineer only when BOTH ALB errors are high AND ECS CPU is high.

```
// CDK: Composite alarm — alert only when both ALB 5xx AND ECS CPU are high
Alarm albAlarm = Alarm.Builder.create(this, "AlbAlarm") .alarmName("ShopFlow-ALB-5xx-High")
    .metric(albMetric).threshold(100).evaluationPeriods(3) .build();
Alarm cpuAlarm = Alarm.Builder.create(this, "CpuAlarm") .alarmName("ShopFlow-ECS-CPU-High")
    .metric(cpuMetric).threshold(85).evaluationPeriods(2) .build();
// Composite: page only when BOTH alarms are in ALARM state simultaneously
CompositeAlarm outageAlarm = CompositeAlarm.Builder.create(this, "OutageAlarm")
    .compositeAlarmName("ShopFlow-Outage-Critical") .alarmRule(AlarmRule.allof(albAlarm, cpuAlarm))
    // AND logic
    .alarmDescription("CRITICAL: ALB 5xx high AND ECS CPU high") .build();
outageAlarm.addAlarmAction(new SnsAction(oncallSnsTopic));
```

14-17

Automated Remediation — Alarm to Lambda

CloudWatch Alarms can trigger SNS, which triggers Lambda for automated remediation. ShopFlow uses this to automatically increase DynamoDB write capacity when throttling is detected.

```
// Lambda: Auto-remediate DynamoDB throttling by increasing WCU 50%
@LambdaHandler public class DynamoDBThrottleRemediator {
    private final DynamoDbClient dynamo = DynamoDbClient.create();

    public void handleRequest(SNSEvent event, Context ctx) {
        DescribeTableResponse desc = dynamo.describeTable(
            DescribeTableRequest.builder().tableName("shopflow-orders").build());
        long currentWcu = desc.table().provisionedThroughput().writeCapacityUnits();
        long newWcu = (long)(currentWcu * 1.5);
        // increase 50%
        dynamo.updateTable(
            UpdateTableRequest.builder().tableName("shopflow-orders")
                .provisionedThroughput(ProvisionedThroughput.builder()
                    .readCapacityUnits(desc.table().provisionedThroughput().readCapacityUnits())
                    .writeCapacityUnits(Math.min(newWcu, 10000)) // cap at 10000 WCU
                .build()).build());
        ctx.getLogger().log("Increased WCU: " + currentWcu + " -> " + newWcu);
    }
}
```

14-18

CloudWatch Synthetics — Outside-In Monitoring

CloudWatch Synthetics runs scheduled canary scripts that test your application from outside — just like a real customer. If a canary fails, CloudWatch Alarms fire before any real customer is affected.

Canary Type	What It Tests	ShopFlow Use Case	Schedule
Heartbeat Monitor	Loads a URL and checks status code and page load time	Homepage loads with 200 OK and 1.8s load time	Every 5 minutes
API Canary	Calls a REST API endpoint with expected POST/PUT/DELETE test order payload	Every 10 mins	Created within 2s
Broken Link Checker	Crawls a page and checks all links return 200	Product detail page has no broken image links or missing API calls	Every hour
Visual Monitoring	Takes a screenshot and compares to a baseline	Checkout page layout matches expected design (no broken CSS)	Every 15 minutes

INFO Canary vs Real User Monitoring

Synthetics canaries are synthetic — they run on a schedule regardless of real user traffic. CloudWatch RUM uses JavaScript injected into the browser to collect actual customer performance data (Core Web Vitals: LCP, FID, CLS). ShopFlow uses canaries for uptime (is the site up?) and RUM for performance (is the site fast for real users in Europe?).

14-19

Observability Cost Analysis

ShopFlow's observability stack costs approximately \$180/month at 1M MAU.

Component	Monthly Cost	Key Cost Driver	Optimization
CloudWatch Metrics — AWS Service	Free	AWS service metrics are free	No change needed
CloudWatch Metrics — Custom	\$6	20 custom metrics x \$0.30/metric/month	Low cost, high value
CloudWatch Logs — Ingest	\$40	8 GB/month JSON logs at \$0.50/GB	Set INFO in prod; avoid DEBUG (\$40/month)
CloudWatch Logs — Storage	\$8	30-day retention x 8 GB at \$0.03/GB	Archive to S3 Glacier after 30 days (\$0.004/GB)
CloudWatch Logs Insights	\$5	Queries scan log data at \$0.005/GB	Narrow time ranges to reduce scanned data
X-Ray Traces	\$25	5% of 100M req/month = 5M traces at \$5/M	Acceptable; 1% sampling on Black Friday peak
Container Insights	\$30	60 tasks x 20 metrics/task x \$0.50/M	Disable in dev/staging environments
CloudWatch Alarms	\$1	12 alarms x \$0.10/alarm/month	Negligible
TOTAL	~\$115/month	Logs and Container Insights dominate	Tune log level; set retention policies

14-20

Q&A — Common Questions

Question	Answer
How do I find which ECS task handled a specific request?	Use Logs Insights filter by traceId (from X-Ray) and search across all log streams. The trace ID is in the log message.
CloudWatch Alarm keeps going in and out of ALARM state. Why?	Use CloudWatch Alarms (require multiple conditions). Also set evaluation period > 1 (e.g., 3 out of 4 periods must be in ALARM state).
X-Ray shows long "dwell time" between service calls. What does this mean?	Dwell time is the time a request spends waiting between service calls (e.g., SQS queue wait). Look for long gaps in the trace.
Can I correlate a specific customer complaint with a request?	Yes, by attaching a userId as an X-Ray annotation: seg.putAnnotation("userId", userId). Then filter in CloudWatch Logs.
How do I monitor Lambda cold starts?	CloudWatch Logs: filter message like /Init Duration/ (Lambda logs cold start duration). Also use CloudWatch Metrics for Duration.

14-21

Best Practices Summary

Practice	Details
Use structured JSON logging	JSON logs are parsed automatically by Logs Insights — query by field name (filter orderId = "X") instead of message content.
Set log retention policies	Default CloudWatch Logs retention is NEVER EXPIRE. Set: 30 days for application logs, 90 days for infrastructure logs.
Alarm on business metrics, not just infrastructure	Infrastructure alarms catch infrastructure failures. Custom metrics like OrdersPlaced < 10 in 5 minutes catch business issues.
Use p99 latency, not average	Average latency hides outliers. p99 (99th percentile) tells you what 1 in 100 customers experiences.

Practice	Details
Correlate CloudWatch and X-Ray with trace IDs	Inject X-Ray trace ID into Spring Boot log MDC. One Logs Insights query can find all logs for any fa
Test alarms in staging	Create alarms in staging with same thresholds as production. Intentionally trigger them to verify the
Use Composite Alarms to avoid alert fatigue	Single metric alarms cause false pages. Composite alarms (ALB 5xx HIGH AND CPU HIGH) ensur

14-22

CloudWatch Metric Math — Derived Metrics

Metric Math lets you combine raw metrics into derived metrics directly in CloudWatch — no extra code or Lambda needed. ShopFlow uses Metric Math to calculate error rates, cache hit ratios, and order conversion rates on dashboards.

```
// CloudWatch Metric Math expressions (used in dashboards and alarms) // Example 1: ALB Error
Rate (5xx as % of total requests) // Metric m1 = HTTPCode_ELB_5XX_Count (Sum/1min) // Metric m2
= RequestCount (Sum/1min) // Math expression: (m1 / m2) * 100 // Result: "Error Rate %" — shown
as line graph on dashboard // Example 2: ElastiCache Hit Rate // Metric m1 = CacheHits
(Sum/1min) — from AWS/ElastiCache // Metric m2 = CacheMisses (Sum/1min) — from AWS/ElastiCache
// Math expression: (m1 / (m1 + m2)) * 100 // Result: "Cache Hit Rate %" — should stay above 90%
for ShopFlow product cache // Example 3: DynamoDB Throttle Rate // Metric m1 = ThrottledRequests
(Sum/1min) // Metric m2 = SuccessfulRequestLatency SampleCount (Sum/1min) = total requests //
Math expression: (m1 / m2) * 100 // Result: "DynamoDB Throttle Rate %" — alarm when > 0.5% //
CDK: Create alarm on Metric Math expression MathExpression errorRate =
MathExpression.Builder.create().expression("(m1 / m2) * 100") .usingMetrics(Map.of( "m1",
Metric.Builder.create().namespace("AWS/ApplicationELB") .metricName("HTTPCode_ELB_5XX_Count")
.statistic("Sum").period(Duration.minutes(1)).build(), "m2", Metric.Builder.create()
.namespace("AWS/ApplicationELB") .metricName("RequestCount")
.statistic("Sum").period(Duration.minutes(1)).build())) .label("ALB 5xx Error Rate (%)")
.period(Duration.minutes(1)) .build(); Alarm.Builder.create(this, "ErrorRateAlarm")
.alarmName("ShopFlow-ErrorRate-High") .metric(errorRate) .threshold(1.0) // alarm when error
rate > 1% .evaluationPeriods(5) .build();
```

14-23

VPC Flow Logs — Network Traffic Analysis

VPC Flow Logs capture information about IP traffic going to and from network interfaces in your VPC. ShopFlow enables them for security analysis and to diagnose connectivity issues (e.g., why can't an ECS task reach Aurora?).

Field	Description	ShopFlow Use Case
srcaddr	Source IP address	ECS task private IP (source); external client IP (inbound traffic)
dstaddr	Destination IP address	Aurora private IP (ECS -> DB); ElastiCache IP (ECS -> Redis)
dstport	Destination port	5432 = Aurora PostgreSQL; 6379 = Redis; 443 = HTTPS outbound
action	ACCEPT or REJECT	REJECT = security group blocking the connection; diagnose conne
bytes	Bytes transferred	Identify chatty services; detect unexpected large data transfers
log-status	OK, NODATA, SKIPDATA	NODATA = no traffic during capture window; SKIPDATA = logs wer

```
// CDK: Enable VPC Flow Logs to CloudWatch Logs LogGroup flowLogGroup =
LogGroup.Builder.create(this, "FlowLogs") .logGroupName("/vpc/shopflow-prod/flow-logs")
.retention(RetentionDays.ONE_WEEK) // flow logs are voluminous; 7 days is enough .build();
FlowLog.Builder.create(this, "VpcFlowLog") .resourceType(FlowLogResourceType.fromVpc(vpc))
.destination(FlowLogDestination.toCloudWatchLogs(flowLogGroup))
.trafficType(FlowLogTrafficType.REJECT) // only log REJECTED traffic for security analysis
.build(); // Logs Insights query: Find rejected connections to Aurora (security group issue) //
fields @timestamp, srcaddr, dstaddr, dstport, action // | filter action = "REJECT" and dstport =
5432 // | stats count() by srcaddr, dstaddr // | sort count() desc
```

14-24

AWS Health Dashboard and CloudTrail Audit Logs

Two additional AWS monitoring services complement CloudWatch: AWS Health (tells you when AWS itself has issues affecting your resources) and CloudTrail (records every API call made to your AWS account).

Service	Purpose	ShopFlow Use Case	Key Difference from CloudWatch
AWS Health Dashboard	Shows AWS service disruptions and scheduled maintenance	Reduced EventBridge traffic for non-critical events	CloudWatch Logs not integrated
AWS CloudTrail	Records every IAM API call (who did what, when)	At-disk log; who deleted? Deleted AWS IAM	CloudTrail logs are not searchable

```
// CDK: CloudTrail for ShopFlow account auditing Trail.Builder.create(this, "CloudTrail")
.trailName("shopflow-audit-trail") .bucket(auditBucket) // store trail in S3 (required)
.includeGlobalServiceEvents(true) // include IAM, STS global events .isMultiRegionTrail(false)
// single region is sufficient for ShopFlow .sendToCloudWatchLogs(true)
.cloudWatchLogsGroup(LogGroup.Builder.create(this, "TrailLogs")
.logGroupName("/cloudtrail/shopflow-prod") .retention(RetentionDays.ONE_YEAR) // compliance: 1
year minimum .build()).build(); // EventBridge: alert when anyone deletes a DynamoDB table
Rule.Builder.create(this, "DynamoDeleteAlert") .eventPattern(EventPattern.builder()
.source(List.of("aws.dynamodb"))) .detailType(List.of("AWS API Call via CloudTrail"))
.detail(Map.of("eventName", List.of("DeleteTable"))) .build()) .targets(List.of(new
SnsTopic(securityAlertTopic))) .build();
```

Black Friday Observability Checklist

One week before Black Friday, ShopFlow's SRE team runs through this observability readiness checklist to ensure the monitoring stack can handle 10x traffic.

Check	Normal State	Black Friday Action	Status
Dashboard ready	ShopFlow-Production-Ops dashboard deployed	Check and deploy ShopFlow-BlackFriday dashboard	Do 2 weeks before
Alarm thresholds reviewed	ALB 5xx > 100/min	Lower to > 50/min for Black Friday (more sensitive)	Do 1 week before
X-Ray sampling for /orders	100% sampling	Keep 100% for /orders — checkout is the most critical	Keep as-is
X-Ray sampling for all other	5% sampling	Reduce to 1% — 10x traffic would generate 10x traces	Set and deploy before
Container Insights verified	Enabled in prod	Verify per-task metrics appearing; test Logs Insights	Do 1 week before
On-call rotation confirmed	Standard 24h shifts	Two SREs on-call simultaneously during Black Friday (6 AM - 11 PM)	Do 1 week before
Runbook ready	General runbook	Specific runbook for: DynamoDB throttling, ECS CPU spike	2 weeks before
SNS -> PagerDuty tested	Monthly test	Full-chain test: trigger alarm -> SNS -> PagerDuty -> Ops	Do 1 week before

CloudWatch Anomaly Detection — Smart Alarms

Standard CloudWatch Alarms use a fixed threshold (e.g., CPU > 80%). Anomaly Detection uses machine learning to learn the normal pattern of a metric over time (day of week, time of day) and alerts when the metric deviates unexpectedly. This is perfect for ShopFlow's order volume, which peaks on weekends and evenings.

```
// CDK: Anomaly Detection alarm for OrdersPlaced (adapts to weekly patterns) Metric
ordersMetric = Metric.Builder.create().namespace("ShopFlow/Orders")
.metricName("OrdersPlaced").statistic("Sum").period(Duration.minutes(5)).build(); // Anomaly
Detection trains on 2+ weeks of data to learn patterns MathExpression anomalyBand =
MathExpression.Builder.create().expression("ANOMALY_DETECTION_BAND(m1, 2)") // 2 = standard
deviations .usingMetrics(Map.of("m1", ordersMetric)).label("Orders Expected Band").build();
// This alarm fires when OrdersPlaced is BELOW the expected band // (i.e., orders are
unexpectedly low – checkout may be broken) Alarm anomalyAlarm = Alarm.Builder.create(this,
"OrderAnomalyAlarm").alarmName("ShopFlow-Orders-AnomalyLow").metric(ordersMetric)
.comparisonOperator(ComparisonOperator.LESS_THAN_LOWER_OR_GREATER_THAN_UPPER_THRESHOLD)
.threshold(0) // threshold ignored when using anomaly detection .evaluationPeriods(3) .build();
// Benefit: on a Tuesday morning when orders are naturally low (100/5min), // a fixed threshold
of < 10 would not alert. Anomaly Detection knows // Tuesday 9 AM should have ~90-110 orders/5min
and alerts if it drops to 40.
```

The CloudWatch Embedded Metrics Format (EMF) lets you publish custom metrics by embedding them directly in your CloudWatch Logs JSON output — with no extra API calls. CloudWatch automatically extracts metrics from EMF-formatted log lines.

```
// Spring Boot: Publish metrics via Embedded Metrics Format (EMF) // No CloudWatch SDK needed —
just log a specially formatted JSON line import
software.amazon.cloudwatch.emf.logger.MetricsLogger; import
software.amazon.cloudwatch.emf.model.DimensionSet; import
software.amazon.cloudwatch.emf.model.Unit; @Component public class ShopFlowEMF { // Call in
OrderService after each successful order public void logOrderPlaced(String category, double
valueUsd, long durationMs) { MetricsLogger logger = new MetricsLogger(); // Dimensions (become
CloudWatch metric dimensions) logger.putDimensions(DimensionSet.of("Category", category)); //
Metrics (extracted from log and stored in CloudWatch) logger.putMetric("OrdersPlaced", 1,
Unit.COUNT); logger.putMetric("OrderValueUSD", valueUsd, Unit.NONE);
logger.putMetric("OrderDurationMs", durationMs, Unit.MILLISECONDS); // Properties (stored in
log but NOT extracted as metrics) logger.putProperty("orderId", orderId);
logger.putProperty("userId", userId); logger.setNamespace("ShopFlow/Orders"); logger.flush();
// writes JSON log line with _aws.CloudWatchMetrics field } } // The log line looks like: // {
// "_aws": { "Timestamp": 1234567890, "CloudWatchMetrics": [{ // "Namespace":
"ShopFlow/Orders", // "Dimensions": [{"Category"}], // "Metrics":
[{"Name":"OrdersPlaced","Unit":"Count"}, {"Name":"OrderValueUSD"}] // }]}, // "Category":
"Electronics", "OrdersPlaced": 1, "OrderValueUSD": 299.99, // "orderId": "ORD-12345" // stored
in log but not as metric // } // CloudWatch extracts OrdersPlaced and OrderValueUSD as real
metrics automatically
```

INFO EMF vs putMetricData

EMF is preferred for Lambda and ECS because: (1) no extra CloudWatch API call (reduce latency and cost), (2) metrics and logs are correlated — one log line gives both the metric value and context (orderId, userId), (3) EMF metrics count against the same free CloudWatch quota. Use putMetricData directly only when you need synchronous confirmation that the metric was received.

ServiceLens combines CloudWatch metrics, logs, and X-Ray traces into a single correlated view. From the ServiceLens Service Map, you can click any node, see its metrics and logs, and drill into specific traces — all without switching between Console sections.

■ Use CloudWatch ServiceLens to Debug a Latency Spike

1. CloudWatch -> ServiceLens -> Service Map
2. Set time range: Last 30 minutes (when the latency spike occurred)
3. Find the orange node — this is the service with high latency or errors
4. Click "order-svc" node -> right panel shows: p50/p90/p99 latency + error rate
5. Click "View traces" -> filter for slow traces (duration > 2s)
6. Click the slowest trace -> Trace Map shows every service call with timing
7. Click "DynamoDB" segment in the trace -> see the throttling error annotation
8. Click "View logs for this trace" -> Logs Insights opens with trace ID pre-filled
9. All application log lines for this order request appear — find the root cause error message

SUCCESS ServiceLens Workflow

The power of ServiceLens is the correlation: customer complains about slow checkout -> ServiceLens -> find the slow trace -> identify DynamoDB throttling -> jump to logs for that trace -> see the full stack trace with orderId. This investigation used to take 30-60 minutes across multiple tools; ServiceLens reduces it to 5 minutes.

14-29

Chapter Summary — ShopFlow Observability Stack

You now have a complete observability stack for ShopFlow. Here is a summary of what each tool does and when to use it.

Question	Tool to Use	ShopFlow Example
Is the site up right now?	CloudWatch Alarms + Synthetics	Canary tests /health every 1 minute; alarm if canary fails
How many orders in the last hour?	CloudWatch Metrics — custom named metric	OrderRateShopFlowOrdersH from ShopFlow/Orders namespace
Why did order-svc get slow at 2 PM?	AWS X-Ray -> Traces -> filter by time	Find the slowest trace; identify which subsegment (DynamoDB, Aurora)
Which customer got the 500 error?	CloudWatch Logs Insights -> filter by error => ERROR	X-Ray orderId = "ORD-12345"
Is ECS CPU growing over the last week?	CloudWatch Metrics -> CPU graph	Thresholding range Auto Scaling if consistently > 70%
Who deleted the DynamoDB table?	AWS CloudTrail -> Event history -> filter by event name	Select table, find exact timestamp of the API call
Is AWS itself having an issue (not my app)?	AWS Health Dashboard -> Event log	Filter by service (Amazon ECS) and region (us-east-1); see if AWS is having an issue

14-30

Monitoring as Code — CDK Observability Construct

ShopFlow encapsulates all monitoring configuration in a reusable CDK construct. Every ECS service uses this construct to automatically get alarms, dashboards, and X-Ray tracing without duplicating configuration.

```
// Reusable CDK construct: ShopFlowServiceMonitor public class ShopFlowServiceMonitor extends Construct { public ShopFlowServiceMonitor(Construct scope, String id, String serviceName, FargateService ecsService, ITopic alertTopic, Dashboard dashboard) { super(scope, id); // 1. CPU alarm Alarm cpuAlarm = ecsService.metricCpuUtilization().createAlarm(this, "CpuAlarm", CreateAlarmOptions.builder().alarmName("ShopFlow-" + serviceName + "-CPU").threshold(80).evaluationPeriods(2).build()); cpuAlarm.addAlarmAction(new SnsAction(alertTopic)); // 2. Memory alarm ecsService.metricMemoryUtilization().createAlarm(this, "MemAlarm", CreateAlarmOptions.builder().alarmName("ShopFlow-" + serviceName + "-Memory").threshold(85).evaluationPeriods(3).build()).addAlarmAction(new SnsAction(alertTopic)); // 3. Add service CPU to shared dashboard dashboard.addWidgets(GraphWidget.Builder.create().title(serviceName + " CPU + Memory").left(List.of(ecsService.metricCpuUtilization())).right(List.of(ecsService.metricMemoryUtilization())).build()); } } // Usage: one line per service adds full monitoring new ShopFlowServiceMonitor(this, "ProductSvcMonitor", "product-svc", productFargateService, alertTopic, dashboard); new ShopFlowServiceMonitor(this, "OrderSvcMonitor", "order-svc", orderFargateService, alertTopic, dashboard); new ShopFlowServiceMonitor(this, "CartSvcMonitor", "cart-svc", cartFargateService, alertTopic, dashboard);
```

14-31

Incident Response Playbook — ShopFlow Outage Protocol

When a CloudWatch Alarm fires and PagerDuty pages the on-call SRE, they follow this structured playbook. Every step uses the observability tools from this chapter.

Step	Action	CloudWatch/X-Ray Tool	Decision
1. Acknowledge	Accept PagerDuty alert; open ShopFlow ProductionOps dashboard	CloudWatch ProductionOps	Confirm alarm is real (not a data gap or fluke)
2. Classify	Determine scope: single service, AWS health, or all services	AWS Health Dashboard; CloudWatch	ECS/S3 Health Checks OK; All services up -> investigate us-east-1 -> escalate to on-call AWS
3. Locate	Find which service is causing the issue	ServiceLens Service Map	Flashed red on X-Ray -> issue may be data layer (Aurora, DynamoDB)

Step	Action	CloudWatch/X-Ray Tool	Decision
4. Trace	Find slowest/failing traces in the -5m window	X-Ray Traces view filter by error=true	Note the specific subsegment that is slow or erroring
5. Log	Get application error message	CloudWatch Logs Insights: filter by <code>level Java exception</code>	Identify Java exception, error message, and stack trace
6. Remediate	Fix the immediate issue (restart, scale, etc.)	CloudWatch Alarms increase capacity; Update service in DynamoDB console	2-3 minutes
7. Communicate	Update status page and notify stakeholders	Slack + PagerDuty + Slack + status page	Update every 15 minutes during active incident
8. Post-mortem	Write blameless post-mortem	CloudWatch Metrics to quantify impact; X-Ray to follow why, detect time	time, fix time, customers affected